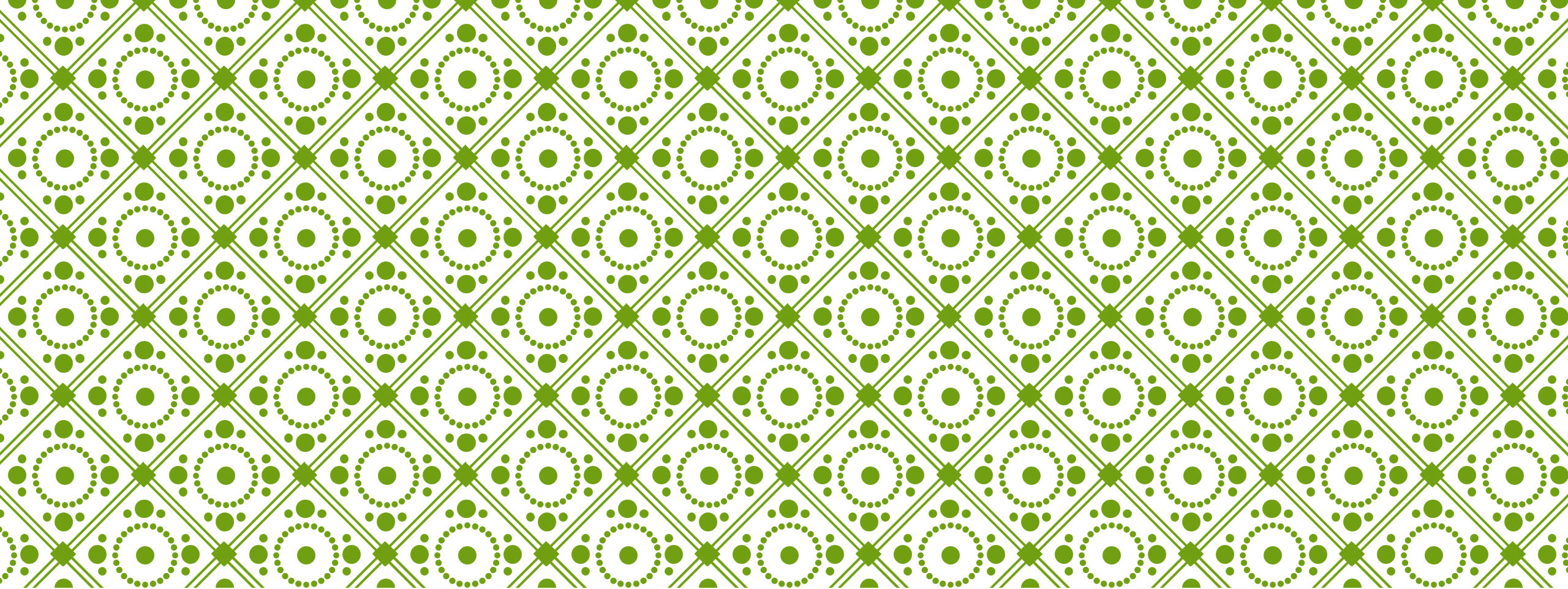




AI2002 – ARTIFICIAL INTELLIGENCE

SPRING 2024 - LECTURE 04-06

Presented By: Mr. Sandesh
Kumar

AGENTS AND ENVIRONMENTS (1/2)

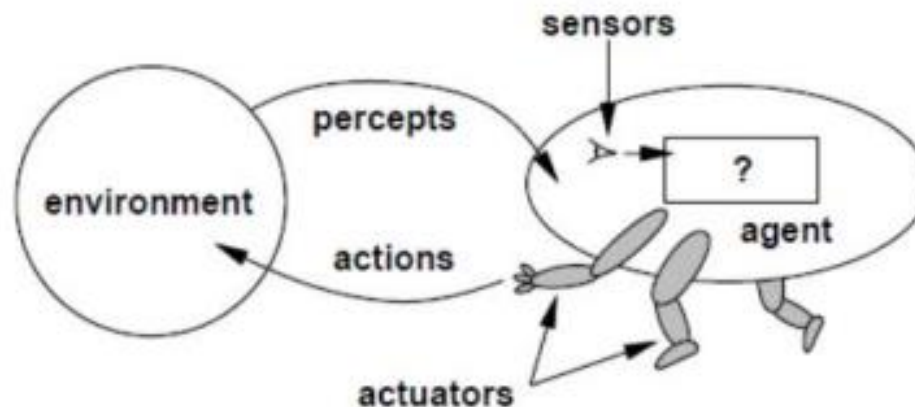
- An **agent** is anything that can be viewed as **perceiving** its **environment** through **sensors** and acting upon that environment through **actuators**.
- The term **percept** to refer to the agent's perceptual inputs at any given instant.
- An agent's **percept sequence** is the complete history of everything the agent has ever perceived.
- An agent's behaviour is described by the **agent function** that maps any given percept sequence to an action.
 - The agent function for an artificial agent will be implemented by an **agent program**.

AGENTS AND ENVIRONMENTS (2/2)

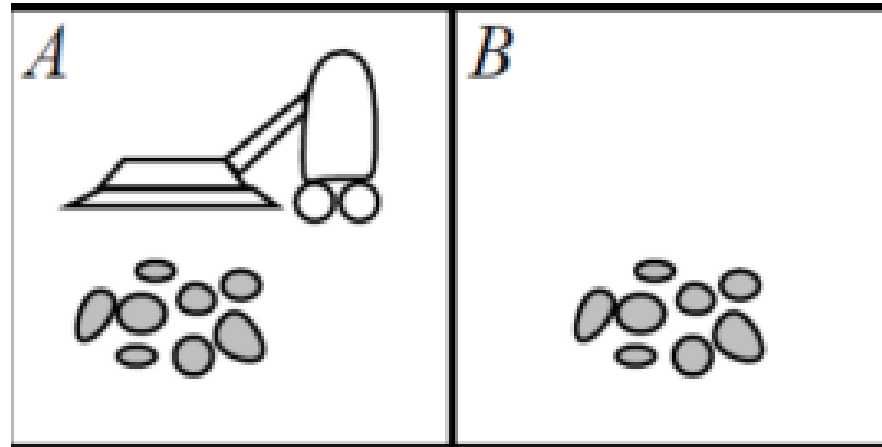
- **Agents** include humans, robots, softbots (software agents), thermostats, etc.
- The **agent function f** maps from **percept histories** to **actions**:

$$f: P^* \rightarrow A$$

- The **agent program** runs on the physical architecture to produce f



VACUUM-CLEANER WORLD WITH TWO LOCATIONS



- **Percepts:** location and contents, e.g. [A; Dirty]
- **Actions:** Left, Right, Suck, NoOp

A VACUUM-CLEANER AGENT

- Partial tabulation of a simple agent function for the vacuum-cleaner world

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

- Various vacuum-world agents can be defined by filling in the right-hand column in various ways.
 - What is the right way to fill out the table?
 - In other words, what makes an agent good or bad, intelligent or stupid?

GOOD BEHAVIOUR: THE CONCEPT OF RATIONALITY

- A **rational agent** is one that does the *right thing* -> What is the *right thing*?
 - A sequence of actions causes the environment to go through a sequence of states.
 - If the sequence is desirable, then the agent has performed well.
- The notion of desirability is captured by a **performance measure** that evaluates any given sequence of **environment states**.
- A **rational agent** chooses whichever action maximizes the *expected value* of the **performance measure** given the *percept sequence*

PERFORMANCE MEASURE

- Consider the vacuum cleaner agent, we might propose to measure performance by the amount of dirt cleaned up in a single eight-hour shift. With a rational agent, what you ask for is what you get.
- A rational agent can maximize this performance measure by cleaning up the dirt, then dumping it all on the floor, then cleaning it up again and so on.
- A more suitable performance measure would reward the agent for having a clean floor. i.e. one point could be awarded for each clean square at each time step.
- *it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.*

RATIONALITY: OMNISCIENCE, LEARNING AND AUTONOMY

- An **omniscient (perfect)** agent knows the actual outcome of its actions and can act accordingly. But perfection is impossible in reality.
- **Rationality** is NOT the same as **perfection**.
 - **Rationality** maximizes *expected performance*, while **perfection** maximizes *actual performance*.
- A rational agent not only to **gather information (exploration)** but also to **learn** as much as possible from what it perceives.
- An agent relies on the prior knowledge of its designer rather than on its own percepts, we say that the agent lacks **autonomy**. A rational agent should be **autonomous**.

TASK ENVIRONMENT: PEAS

- To design a *rational agent*, we must specify the **task environment**.
- The performance measure, the environment, and the agent's actuators, and sensors are grouped as the **task environment**, and called as **PEAS** (Performance measure, Environment, Actuators, and Sensors)

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

- PEAS description of the task environment for an automated taxi

ENVIRONMENT TYPES (1/3)

- The **environment type** largely determines the agent design.
- **Fully observable vs Partially observable:**
 - If an agent's sensor give it access to the complete state of the environment at each point in time, then task environment is called as **fully observable**
 - Fully observable environments are convenient because the agent need not maintain any internal state to keep track of the world.
 - An environment might be **partially observable** because of noisy and inaccurate sensors.
- **Single Agent vs Multi Agent:**
 - One or more agents. Solving a crossword puzzle is a single agent environment.
 - Chess is **competitive** multi agent environment because an agent tries to maximize its performance while minimizing the performance of other agent.
 - Taxi-driving partially **cooperative** multi agent environment because avoiding collisions maximizes all agent's performances.

ENVIRONMENT TYPES (2/3)

➤ **Deterministic vs Stochastic:**

- if the next state of the environment is completely determined by the current state and the action executed by the agent, then the environment is **deterministic** otherwise, it is **stochastic**.

➤ **Episodic vs Sequential:**

- In an **episodic task environment** the agent's experience is divided into atomic episodes. In each episode the agent receives a percept and then performs a single action. The next episode does not depend on the actions taken in previous episodes.
- In **sequential environment**, on the other hand, the current decision could affect all future decisions. Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences.

➤ **Static vs Dynamic:**

- if the environment can change while an agent is deliberating, then we say the environment is **dynamic** for that agent; otherwise, it is **static**.

➤ **Discrete vs Continuous:**

- The discrete/continuous distinction applies to the *state* of the environment.
- Chess is discrete; Taxi-driving continuous.

ENVIRONMENT TYPES (3/3) — EXAMPLES OF TASK ENVIRONMENTS AND THEIR CHARACTERISTICS

- The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

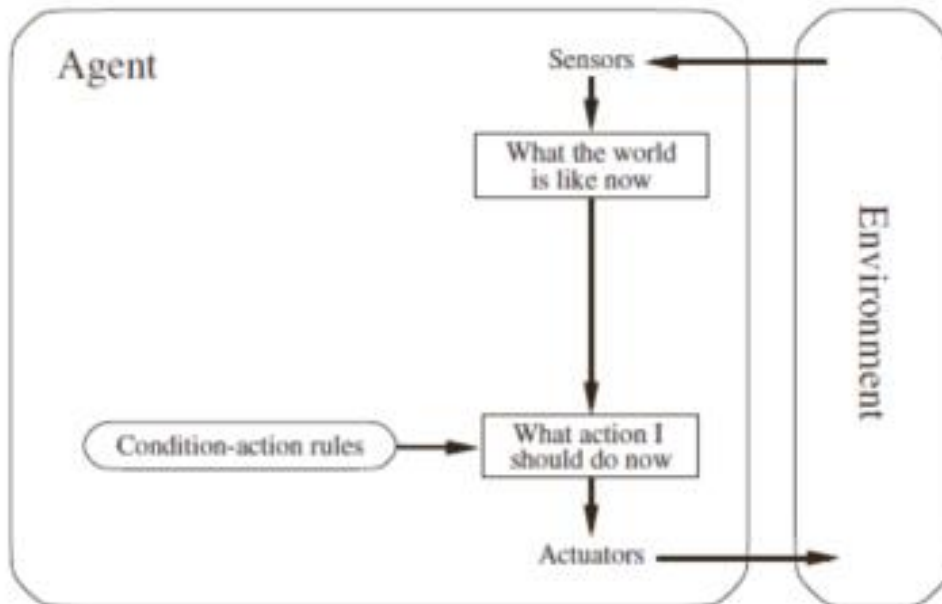
Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

AGENT TYPES

- Four basic types in order of increasing generality:
 - simple reflex agents
 - reflex agents with state
 - goal-based agents
 - utility-based agents
- All these can be turned into learning agents.

SIMPLE REFLEX AGENTS

- **Simple reflex agents** select actions on the basis of the *current* percept, ignoring the rest of the percept history.



```
function SIMPLE-REFLEX-AGENT(percept) returns an action  
persistent: rules, a set of condition–action rules  
  
state ← INTERPRET-INPUT(percept)  
rule ← RULE-MATCH(state, rules)  
action ← rule.ACTION  
return action
```

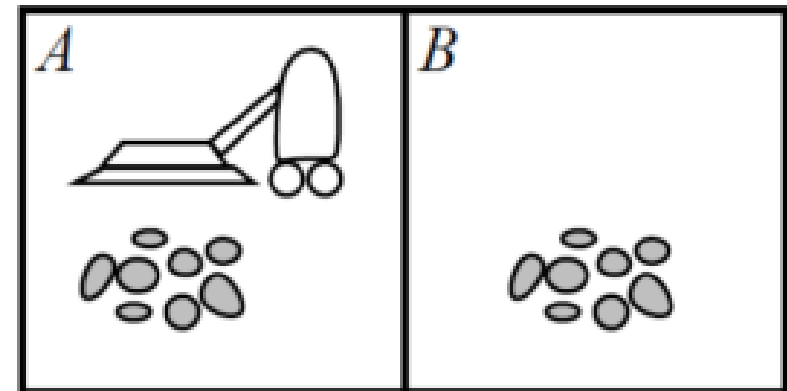
SIMPLE REFLEX AGENT IN THE TWO-STATE VACUUM ENVIRONMENT

function REFLEX-VACUUM-AGENT(*[location, status]*) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

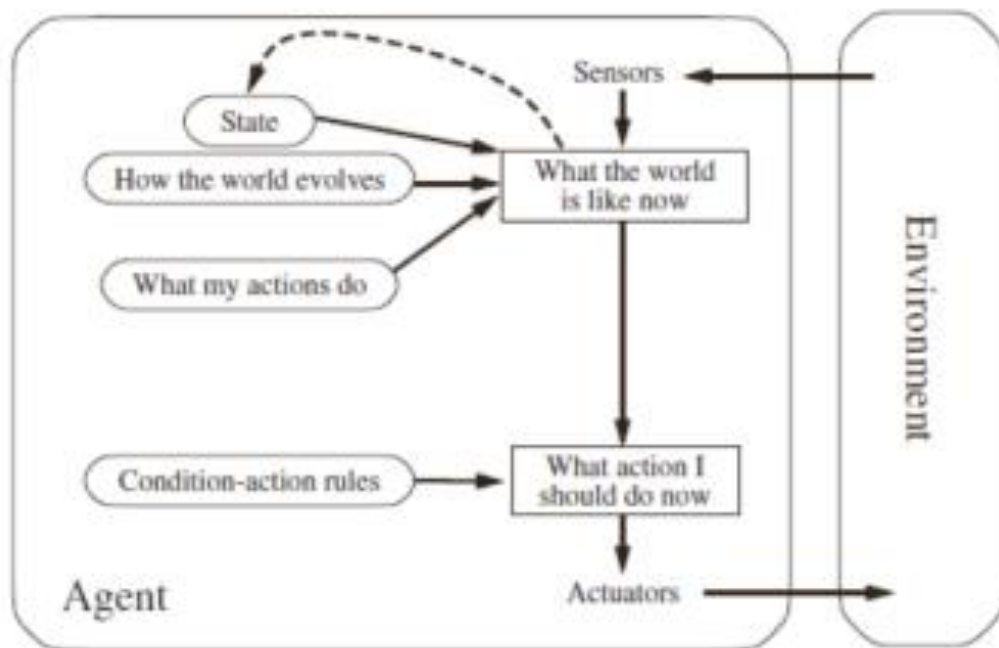
➤ This program implements the agent function in the following table.

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>



MODEL-BASED REFLEX AGENTS

- A **model-based reflex agent** keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent



function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action

persistent: *state*, the agent's current conception of the world state

model, a description of how the next state depends on current state and action

rules, a set of condition-action rules

action, the most recent action, initially none

state ← UPDATE-STATE(*state*, *action*, *percept*, *model*)

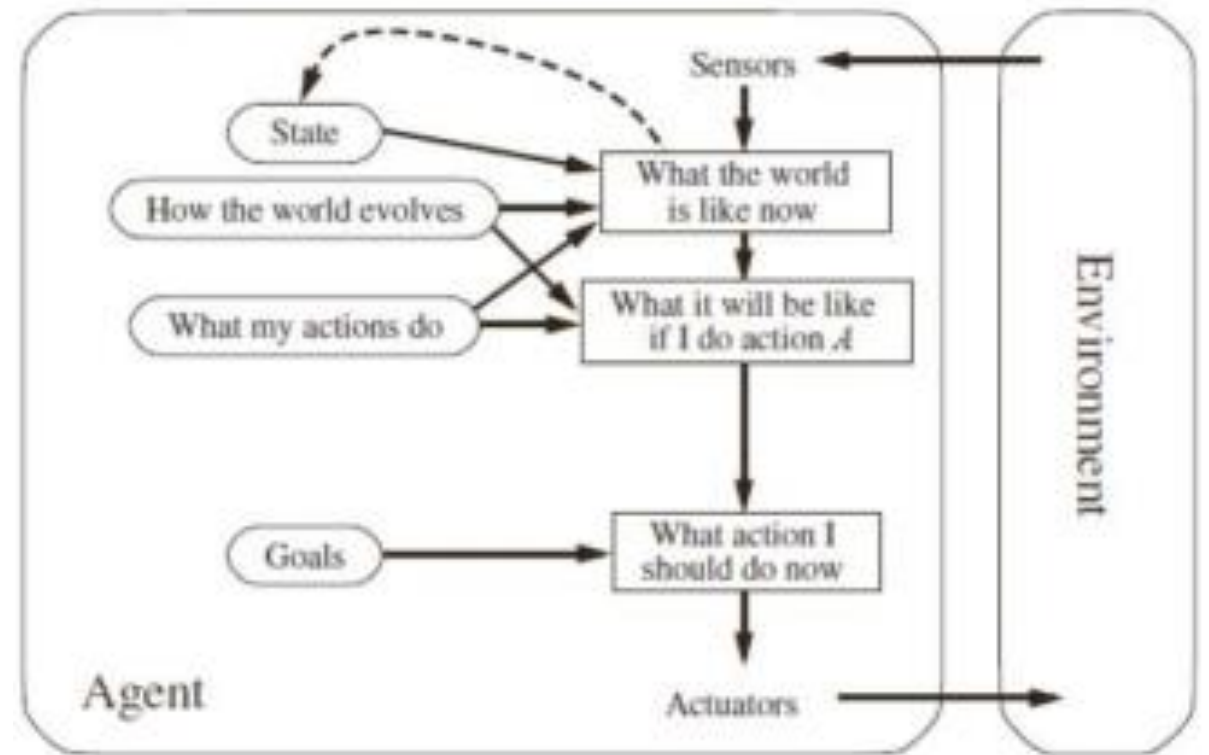
rule ← RULE-MATCH(*state*, *rules*)

action ← *rule*.ACTION

return *action*

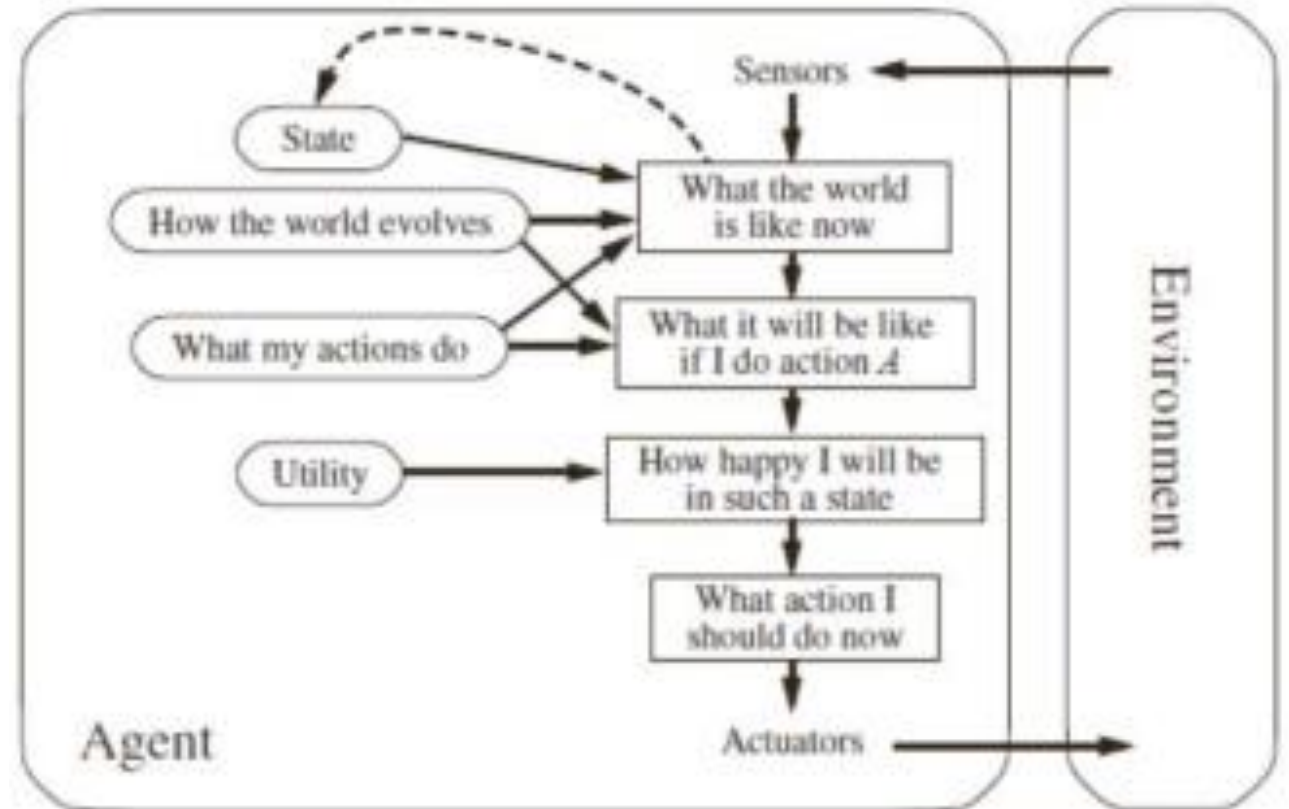
GOAL-BASED AGENTS

- A **goal-based agent** keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.



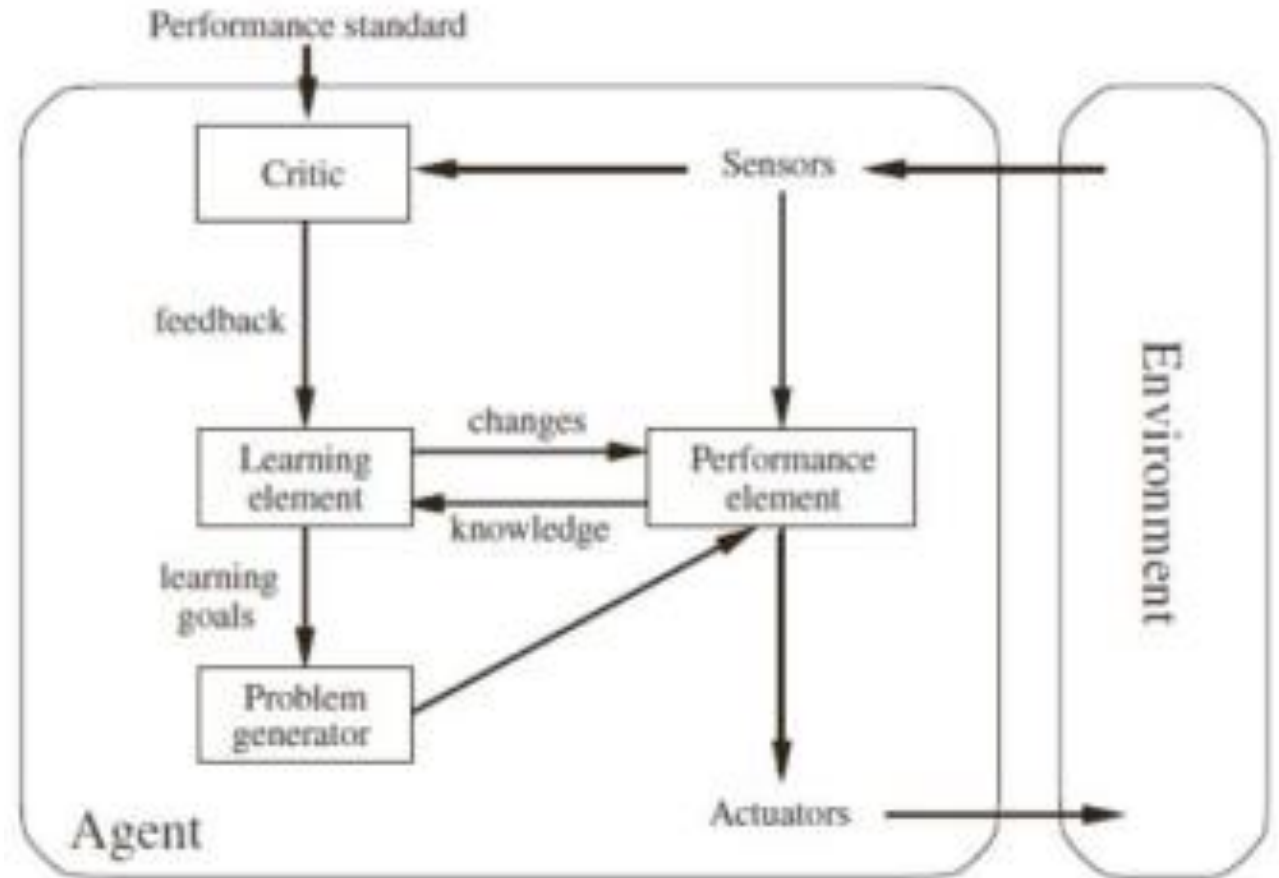
UTILITY-BASED AGENTS

- A **utility-based agent** uses a model of the world, along with a utility function that measures its preferences among states of the world.
- Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.



LEARNING AGENTS

- **learning element** is responsible for making improvements.
- **performance element** is responsible for selecting external actions.
- The learning element uses feedback from **critic** on how agent is doing and determines how performance element should be modified to do better in future.
- **problem generator** is responsible for suggesting actions that will lead to new and informative experiences.



PROBLEM REPRESENTATION IN AI

- One of the core aspects of AI is problem representation, which plays a crucial role in how machines understand and solve complex problems. In simple terms, problem representation refers to the *process of transforming real-world situations or scenarios into a format that can be comprehended and processed by machines.*
- *Representation in AI* is a fundamental concept that enables machines to reason, learn, and make decisions. It involves encoding various aspects and attributes of a problem, such as objects, relationships, constraints, and goals, into a structured format that can be manipulated and analysed by AI algorithms.